

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

ПРОТОКОЛ
лабораторної роботи №1
«Розгортання середовища та порівняльний аналіз мовних моделей»

Виконав(-ла): студент групи AI242
Манкевич Т.О.

Перевірів: ас. Баранов О.М.

Одеса – 2026

Середовище та джерела даних

Експерименти виконано на локальному сервері **Ollama** (localhost:11434; моделі llama3.2:3b та qwen3:4b) і в хмарі через OpenAI-сумісний ендпойнт Google AI Studio (модель серії Gemini Flash). Клієнт — бібліотека openai для Python; скрипти: hello.py, compare.py, benchmark.py, quality.py, sampling.py, запуск усіх кроків — run_all.py. Завдання 1 і тест холодного та теплого старту (завдання 4) виконано окремими скриптами task1_reasoning.py і task4_cold_warm.py напряду через нативний API Ollama (/api/generate).

Усі числа в протоколі взято з логів запусків (out/task2_hello.txt, out/task3_compare.txt, out/task4_benchmark.txt, out/task6_sampling.txt, results.csv, quality.json, консольний вивід task1_reasoning.py і task4_cold_warm.py). Величини, обчислені за цими даними, позначено як розрахункові. Назва хмарної моделі в різних логах відрізняється (gemini-3.5-flash — завдання 3, gemini-3.6-flash — завдання 4, gemini-3.8-flash — завдання 5), бо провайдер оновлював доступні версії; у кожному розділі назву наведено так, як вона записана в логу.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано через task1_reasoning.py: запити до двох локальних моделей — міркувальної qwen3:4b та інструктивної llama3.2:3b — через ендпойнт /api/generate. Спочатку просте фактичне питання «Столиця України? Відповідай одним словом.», потім — задача на міркування з п'ятьма запусками для кожної моделі. Умова задачі (відновлено за відповідями моделей): ручка і зошит разом коштують 110 грн, зошит дорожчий за ручку на 100 грн; потрібно знайти ціну ручки (правильно — 5 грн). eval_count — кількість згенерованих токенів; у qwen3:4b вона включає токени прихованого поля thinking.

Таблиця 1 — Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Киев	Київ.
eval_count	133	5
Час, с	4.44	0.18
Токенів/с	30.7	47.9
Поле thinking	є	немає

Обидві відповіді змістовно правильні, але різняться за формою: qwen3:4b написала «Киев» (російське написання, українською — «Київ»), а llama3.2:3b — «Київ.» із крапкою, хоча просили одне слово. Розрахункове співвідношення: qwen3:4b згенерувала у 26.6 раза більше токенів (133 проти 5) і витратила у ≈ 25 разів більше часу (4.44 проти 0.18 с).

Таблиця 2 — Задача на міркування (5 запусків для кожної моделі)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	3 / 5

Показник	qwen3:4b	llama3.2:3b
eval_count, середнє	1136.6	146.2
eval_count, діапазон	961–1352	89–200
Час, с, середнє	39.87	5.03
Час, с, діапазон	33.17–47.39	3.08–7.83

Автоматичну оцінку скрипта (5/5 і 3/5) перевірено вручну за текстами відповідей: qwen3:4b у всіх запусках отримала 5 грн, llama3.2:3b — 5 грн у запусках 2, 3 і 5 та 10 грн у запусках 1 і 4. Типова помилка llama3.2:3b: відповідь «10 грн» — модель віднімає різницю від суми (110 – 100) і вважає залишок ціною ручки, хоча залишок дорівнює 2х, а не х. Перевірка: при 10 грн зошит коштував би 110 грн, а разом — 120 грн, не 110. Обидві хибні відповіді — найкоротші (89 і 115 токенів), у правильних запусках було 160–200 токенів, тобто модель помилилась там, де не розписала рівняння.

Мовна примітка: у всіх п'яти розв'язках llama3.2:3b трапляються російські слова й форми («Предположим», «Вычтите 100 из обеих частей», «составляет»), а в запуску 5 — навіть англійське «together». qwen3:4b пише українською, але з русизмами й помилками узгодження («Обозначим», «Упрощаємо», «від обидві частини»).

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: У приховане поле thinking. На питання «Столиця України?» qwen3:4b згенерувала 133 токени замість 5; сама відповідь «Київ» займає кілька токенів, тож приблизно 130 токенів пішло на внутрішні роздуми над очевидним фактом, перш ніж модель дала відповідь, яку дала б і без них.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?

Відповідь: Перевага — на багатокрокових задачах (арифметика, логіка, планування), де проміжний результат потрібно тримати в тексті: у задачі про ручку й зошит qwen3:4b дала 5/5 проти 3/5 у llama3.2:3b, але ціною ≈ 8 разів більшого часу (39.87 проти 5.03 с). На простих фактичних питаннях і коротких відповідях режим міркування лише витрачає час і токени: відповідь та сама, а витрати більші у ≈ 25 разів (4.44 проти 0.18 с).

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Міркувальну модель додатково донавчають (зокрема навчанням із підкріпленням) генерувати ланцюжок роздумів (chain-of-thought) перед фінальною відповіддю. Інструктивну вчать одразу видавати готову відповідь без проміжного тексту.

4. Чи означає більша кількість згенерованих токенів у міркувальній моделі, що вона «розумніша» за інструктивну?

Відповідь: Ні. Кількість токенів характеризує обсяг роботи, яку модель собі призначила, а не якість. У табл. 1 133 токени не змінили результату (відповідь та сама, а її написання гірше — «Київ»). У задачі про ручку більше токенів справді збіглося з кращим результатом, але причина — розписаний хід розв'язання, а не кількість токенів як така: llama3.2:3b помилялась саме тоді, коли його пропускала.

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b?

Відповідь: Швидкість генерації в моделях одного порядку: 30.7 проти 47.9 токенів/с у табл. 1 (у qwen3:4b на $\approx 36\%$ нижча; на відповіді з 5 токенів вимір ще й грубий). У задачі про ручку розрахункова швидкість (eval_count / час) становила 26.6–30.2 токенів/с у qwen3:4b і 36.8–37.5 у llama3.2:3b (запуски 2–5); у запуску 1 llama3.2:3b показала лише 11.4 токенів/с, імовірно, через завантаження моделі в пам'ять після qwen3:4b. Швидкість залежить від розміру моделі й обладнання (3 проти 4 млрд параметрів), тому відрізняється помірно, а от загальний час — у ≈ 8 –25 разів, бо qwen3:4b генерує значно більше токенів (1136.6 проти 146.2; 133 проти 5).

Висновок за завданням 1

Експеримент показав, що швидкість генерації та загальний час відповіді не можна плутати. Швидкість генерації характеризує модель і обладнання: у qwen3:4b вона лише на $\approx 36\%$ нижча (30.7 проти 47.9 токенів/с). Загальний час залежить від обсягу роботи, яку модель собі призначила: на фактичному питанні qwen3:4b згенерувала у ≈ 27 разів більше токенів (133 проти 5) і витратила у ≈ 25 разів більше часу — чиста перевитрата, бо відповідь та сама. На задачі про ручку й зошит ситуація протилежна: розписаний хід розв'язання дав 5/5 проти 3/5, а обидві помилки llama3.2:3b («10 грн») припадають на найкоротші відповіді. Практичний висновок: тип моделі варто обирати під тип задачі — режим міркування виправданий для багатокрокових задач і марнотратний для простих фактів.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Запит користувача: «Поясни різницю між стеком і чергою.» (temperature = 0,7). Скрипт hello.py виконано для двох локальних моделей із двома варіантами system-повідомлення: «Ти лаконічний технічний асистент.» та «Відповідай лише українською, максимум двома реченнями.»

Таблиця 3 — Вплив system-повідомлення на відповідь

Модель	System	Довжина, симв.	Мова відповіді	Речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	732	українська, але з чужими словами («мінералів»)	~7	ні: відповідь не лаконічна, зміст хибний

Модель	System	Довжина, симв.	Мова відповіді	Речень	Інструкцію виконано?
llama3.2:3b	укр., 2 речення	200	українська з англ. вкрапленнями («elementsy», «cheriga»)	2	формально так (мова й 2 речення), зміст хибний
qwen3:4b	лаконічний асистент	195	українська (є описка «последній»)	2	так
qwen3:4b	укр., 2 речення	206	українська	2	так

Змістова різниця: llama3.2:3b в обох варіантах не впізнала «стек» і «чергу» як структури даних — у першому випадку описала їх як «рідкий мінеральний матеріал», у другому — як «будівельний конструктивний елемент». qwen3:4b дала правильне пояснення (LIFO / FIFO).

Таблиця 4 — Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	51	251	302
llama3.2:3b	укр., 2 речення	55	62	117
qwen3:4b	лаконічний асистент	53	4071	4124
qwen3:4b	укр., 2 речення	62	907	969

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Обидві формально вклались у два речення (llama3.2:3b — 200 символів, qwen3:4b — 206). Але з system-повідомленням «лаконічний асистент» коротко відповіла лише qwen3:4b (2 речення, 195 символів), тоді як llama3.2:3b видала близько семи «речень» (732 символи). До того ж відповідь llama3.2:3b змістовно хибна в обох варіантах, тому за сукупністю точніше інструкцію виконала qwen3:4b. Натомість llama3.2:3b значно дешевша за токенами (62 проти 907 completion_tokens).

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Повного переходу на іншу мову не було: усі чотири видимі відповіді українською. Проте llama3.2:3b вставляла чужі слова («mineralів», «elementsy», «cheriga»). Якою мовою міркувала qwen3:4b у прихованому полі thinking, з логів не видно.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: Зріс: у llama3.2:3b з 51 до 55 (+4), у qwen3:4b з 53 до 62 (+9). Друге system-повідомлення довше за перше, тому додає токени. Різниця між моделями (+4 і +9), імовірно, пояснюється різними токенизаторами: той самий текст розбивається на різну кількість токенів.

4. Чому в qwen3:4b completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: qwen3:4b — міркувальна модель: перед відповіддю вона генерує приховане поле thinking, яке теж рахується в completion_tokens. При видимій відповіді у ~200 символів вона витратила 4071 і 907 токенів. У llama3.2:3b completion_tokens (251 і 62) відповідають лише видимому тексту.

Відповіді від чат-бота на питання (крок 7)

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Це залежить від обсягу й якості інструктивного донавчання, розміру моделі та мовного складу навчальних даних. У наших запусках llama3.2:3b порушила вимогу лаконічності й вставляла чужі слова, а qwen3:4b — ні.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

System задає роль і правила, і моделі під час донавчання прищеплюють звичку ставити його вище за user. Але це навчена схильність, а не жорстке правило: при конфлікті з сильним патерном чи конкретним запитом користувача модель може system проігнорувати.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Бо частина вихідних токенів іде на приховані міркування (thinking), які генеруються перед відповіддю навіть для простого запиту й тарифікуються як вихідні.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Однозначної відповіді немає: багато моделей віддають перевагу system, але результат залежить від конкретної моделі та формулювань, тому поведінку слід перевіряти експериментально, а критичні вимоги — контролювати кодом.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі моделі мають менше параметрів і бачили менше україномовних даних, тому гірше утримують мову й частіше «зісковзують» у суміш слів із різних мов — саме це видно у відповідях llama3.2:3b.

Висновок за завданням 2

System-повідомлення реально керує поведінкою моделі: зміна одного рядка зменшила completion_tokens у llama3.2:3b з 251 до 62 (на 75 %), а в qwen3:4b — з 4071 до 907 (на 78 %); при цьому скоротились і приховані міркування. Проте воно не додає моделі знань: llama3.2:3b з обома варіантами system видала змістовно хибну відповідь, лише в різній формі. Тож system-повідомлення регулює форму й витрати, але не правильність відповіді.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Запит: «Поясни різницю між стеком і чергою.» (temperature = 0,7). Виконано через providers.py (make_client) та compare.py — один і той самий код для локального та хмарного провайдера.

Таблиця 5 — Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.5-flash
Час відповіді, с	14.21	9.10
prompt_tokens	40	15
completion_tokens	268	1042
total_tokens	308	2037
Довжина відповіді, символів	836	3093
Швидкість, токенів/с (розрахунок: completion_tokens / час)	18.9	114.5
Мова відповіді	українська з чужими вкрапленнями («zwischen», «и», «проникає»)	українська
Фактичні помилки	так: відповідь про «світло» замість структур даних	ні

Розрахункова примітка: у хмарній відповіді total_tokens (2037) значно більше за суму prompt_tokens і completion_tokens (1057); різниця у 980 токенів, імовірно, припадає на внутрішні міркування моделі, які провайдер рахує окремо.

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: Логіка make_client() і код compare.py лишилися без змін. Провайдери описані у словнику PROVIDERS у providers.py і відрізняються лише трьома полями: base_url, api_key (для хмари береться з файлу .env) та model. Назву хмарної моделі довелося оновлювати: провайдер прибрав доступ до gemini-2.5-flash (помилка 404), і в логах зустрічаються gemini-3.5-flash, gemini-3.6-flash та gemini-3.8-flash.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна: 3093 символи проти 836. Вона правильно пояснила принципи LIFO і FIFO, навела аналогії (стопка тарілок, черга в магазині), операції Push/Pop та Enqueue/Dequeue і порівняльну таблицю. Локальна llama3.2:3b підмінила тему: писала про «світло та чергу» і стверджувала, що стек — «фізичний об'єкт, який проникає світлом».

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, обидва типи. Фактично відповідь хибна повністю (стек і черга не пов'язані зі структурами даних). Мовні проблеми: вкраплення німецького «zwischen», російського «и», змішання латиниці й кирилиці («проникає», «мінералів» у завданні 2) та вигадані слова («вуглеводневнарадяча»).

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: У нашому вимірі так: 9.10 с проти 14.21 с, попри те, що хмарна згенерувала майже вчетверо більше токенів (1042 проти 268). Швидкість генерації в хмарі становила приблизно 114 токенів/с проти 19 у локальній — потужне серверне обладнання з запасом покриває мережеву затримку. Це одиничний вимір, тому висновок стосується саме цього запуску.

Відповіді від чат-бота на питання (крок 6)

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

OpenAI першим отримав масову аудиторію, і на його API написано величезну кількість клієнтського коду та бібліотек. Іншим провайдерам вигідніше підтримати сумісний ендпоінт, ніж змушувати розробників переписувати інтеграції.

2. Я порівнював локальну модель на 3 млрд параметрів (Llama3.2:3b) і хмарну модель Gemini Flash, розмір якої провайдер не розкриває. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Розмір важливий, бо більше параметрів — більше «місткість» знань, але не єдиний фактор. Якість і склад навчальних даних, інструктивне донавчання й вирівнювання нерідко дають не менший ефект. Тому порівнювати треба на конкретних задачах.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Кластери GPU на боці провайдера генерують токени значно швидше за локальний комп'ютер, і цей вииграш перекидає кілька сотень мілісекунд мережевої затримки, особливо на довгих відповідях.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Запити й відповіді покидають ваш комп'ютер і потрапляють на сервери провайдера, де можуть логуватись або використовуватись для покращення моделей. У комерційних проєктах це знімають договорами, режимами без збереження даних, шифруванням, маскуванням персональних даних, а найчутливіші дані обробляють локальними моделями.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Це обмеження кількості запитів або токенів за одиницю часу. Під час завдання 4 ми самі його зачепили: безкоштовний тариф дозволяв 5 запитів на хвилину, і API повернув помилку 429. Розробники користуються кешуванням, чергами з повторними спробами через паузу (backoff), розподілом навантаження між ключами чи провайдерами або платним тарифом.

Висновок за завданням 3

Єдиний OpenAI-сумісний інтерфейс справді зводить перехід між локальною і хмарною моделлю до зміни конфігурації: код порівняння лишився без змін. У нашому вимірі хмарна модель виявилась і швидшою (9.10 проти 14.21 с), і набагато якіснішою (правильна відповідь проти хибної). Водночас платою є передача даних стороннім серверам і ліміти безкоштовного тарифу, тому вибір провайдера — компроміс між якістю, швидкістю, приватністю та вартістю.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Виконано через benchmark.py: три повтори з медіаною для кожної комбінації «провайдер × промпт», перед вимірами — один розігрівуючий запит, який не враховується. Промпти: короткий — «Столиця Японії?»; середній — «Поясни в одному абзаці, що таке рекурсія.»; довгий — «Напиши інструкцію з 10 пунктів, як налаштувати робоче середовище розробника.» Відповідь отримували потоком (stream), TTFT — час до першого токена.

Таблиця 6 — Результати вимірювання швидкості (results.csv)

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.105	2.484	14.7
локальна	llama3.2:3b	середній	2.089	5.067	80.6
локальна	llama3.2:3b	довгий	2.123	17.363	107.8
хмарна	gemini-3.6-flash	короткий	1.696	1.697	15.9
хмарна	gemini-3.6-flash	середній	9.031	9.452	46.4
хмарна	gemini-3.6-flash	довгий	7.902	13.609	231.6

Швидкість у символах на секунду обчислена скриптом як довжина відповіді, поділена на загальний час (разом із TTFT), тому для коротких відповідей показник малий.

Таблиця 7 — Повторні запуски benchmark.py для локальної моделі (медіани з трьох повторів, с)

Промпт	TTFT, запуск 1	TTFT, запуск 2	TTFT, запуск 3	Всього, запуск 1	Всього, запуск 2	Всього, запуск 3
короткий	2.105	2.097	2.075	2.484	2.420	2.412
середній	2.089	2.105	2.104	5.067	4.310	3.699
довгий	2.123	2.106	2.106	17.363	12.706	11.256

Запуск 1 — повний, його результати збережено в results.csv. Запуски 2 і 3 було перервано помилками хмарного API (429 — вичерпано ліміт безкоштовного тарифу у 5 запитів на хвилину; 404 — модель gemini-2.5-flash більше недоступна), тому для них є лише локальні виміри й, у запуску 2, частина хмарних. Індивідуальні значення трьох повторів усередині запуску скрипт не зберігає — лише медіану. Холодний і теплий старт у benchmark.py окремо не фіксуються (розігрівальний запит у логах не записується), тому їх виміряно окремим скриптом task4_cold_warm.py (табл. 8–9).

Холодний і теплий старт вимірювались окремим скриптом task4_cold_warm.py: промпт «Столиця Японії?», три повтори; перед кожним холодним запитом модель вивантажувалась із пам'яті (keep_alive=0), теплий запит виконувався одразу після нього. Колонки: TTFT — час до першого токена (вимірює клієнт); load — load_duration із відповіді Ollama (завантаження ваг у пам'ять); total — total_duration із відповіді Ollama (серверний час запиту); wall — повний час запиту на боці клієнта.

Таблиця 8 — Холодний і теплий старт локальних моделей: усі виміри (с)

Модель	Старт	TTFT, с	load, с	total, с	wall, с
llama3.2:3b	холодний	6.018	3.886	4.368	6.393
llama3.2:3b	теплий	2.078	0.004	0.386	2.430
llama3.2:3b	холодний	6.726	4.592	5.031	7.074
llama3.2:3b	теплий	2.066	0.004	0.378	2.411
llama3.2:3b	холодний	6.289	4.163	4.598	6.636
llama3.2:3b	теплий	2.085	0.003	0.368	2.422
qwen3:4b	холодний	7.598	5.462	18.469	20.499
qwen3:4b	теплий	2.120	0.004	17.543	19.598
qwen3:4b	холодний	6.001	3.857	16.758	18.795
qwen3:4b	теплий	2.126	0.003	17.574	19.635
qwen3:4b	холодний	5.991	3.830	16.750	18.805
qwen3:4b	теплий	2.116	0.006	17.648	19.688

Таблиця 9 — Холодний і теплий старт: медіани з трьох повторів (розрахунок, с)

Модель	Старт	TTFT, с	load, с	total, с	wall, с
llama3.2:3b	холодний	6.289	4.163	4.598	6.636
llama3.2:3b	теплий	2.078	0.004	0.378	2.422
llama3.2:3b	різниця	+4.211	+4.159	+4.220	+4.214
qwen3:4b	холодний	6.001	3.857	16.758	18.805
qwen3:4b	теплий	2.120	0.004	17.574	19.635
qwen3:4b	різниця	+3.881	+3.853	−0.816	−0.830

Холодний старт збільшує TTFT приблизно втричі: у llama3.2:3b з 2.078 до 6.289 с (+4.211 с, $\times 3.0$), у qwen3:4b з 2.120 до 6.001 с (+3.881 с, $\times 2.8$); значення — медіани. Надбавка майже збігається з load_duration (4.163 і 3.857 с), тоді як у теплому стані load_duration ≈ 0.004 с: модель уже перебуває в пам'яті, і час на завантаження ваг не витрачається.

Два додаткові спостереження. По-перше, у всіх 12 вимірах різниця wall – total (час, який не входить у total_duration Ollama) майже стала — 2.025–2.061 с — і не залежить ні від моделі, ні від холодного чи теплового стану. Саме вона задає нижню межу TTFT ≈ 2.1 с у теплому стані та узгоджується з ≈ 2.1 с у табл. 6. Природу цієї сталої затримки скрипти не з'ясовували (імовірно, вона на боці клієнта або локального з'єднання, а не в моделі). По-друге, у qwen3:4b холодний старт у total і wall не проявився (медіана total 16.758 с у холодному проти 17.574 с у теплому): час запиту визначає приховане thinking — ≈ 17.6 с на питання «Столиця Японії?», у ≈ 46 разів більше, ніж у llama3.2:3b (0.378 с). Різниця між запусками, імовірно, зумовлена різною кількістю thinking-токенів; eval_count у цьому тесті не виводився, тому перевірити це за логом не можна.

Відповіді на питання (крок 6)

1. У якій моделі кращий TTFT і чому саме в неї?

Відповідь: Залежить від довжини промпту. На короткому кращий TTFT у хмарної (1.696 с проти 2.105 с), на середньому й довгому — у локальної (≈ 2.1 с проти 9.031 і 7.902 с). У локальної моделі TTFT майже не змінюється (2.09–2.12 с) — немає мережі й черги. У хмарної він коливається від 1.7 до 9.0 с: до нього входять мережа, черга на сервері й, імовірно, внутрішні міркування моделі перед першим видимим токеном.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Не завжди. На середньому промпті локальна виграє за обома показниками (TTFT 2.089 проти 9.031 с; 80.6 проти 46.4 симв./с). На довгому вона виграє за TTFT, але програє за символами/с (107.8 проти 231.6): хмарна довго «розганяється», але потім генерує вдвічі швидше. На короткому обидва показники трохи кращі в хмарної.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: Короткий: загальний час у хмарної менший (1.697 проти 2.484 с). Середній: перевага в локальної (5.067 проти 9.452 с). Довгий: знову виграє хмарна за загальним часом (13.609 проти 17.363 с) завдяки пропускній здатності, хоча її TTFT утричі гірший (7.902 проти 2.123 с). Тобто з ростом обсягу відповіді значення TTFT для загального часу зменшується, а швидкість генерації — зростає.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Індивідуальні повтори не логувались, але між трьома запусками скрипта (табл. 7) медіани TTFT локальної моделі відрізняються менш ніж на 1.5 %, а загальний час довгої відповіді — від 11.256 до 17.363 с (розкид ≈ 54 %), середньої — від 3.699 до 5.067 с (≈ 37 %). Отже, одного запуску недостатньо: результат залежить від фонового навантаження, тому потрібні повтори й медіана.

5. Чому холодний старт повільніший за теплий?

Відповідь: Через завантаження ваг. За вимірами холодного й теплового старту (табл. 8–9) холодний TTFT становив 6.289 с у llama3.2:3b і 6.001 с у qwen3:4b проти 2.078 і 2.120 с у теплому стані, тобто приблизно втричі більше. Надбавка ≈ 4 с збігається з load_duration (4.163 і 3.857 с): при холодному старті Ollama спершу зчитує ваги моделі з диска в пам'ять, а при теплому модель уже резидентна, і load_duration ≈ 0.004 с. На загальному часі відповіді qwen3:4b ця різниця не проявилась (total ≈ 17 с в обох станах), бо в ньому домінує приховане thinking.

Відповіді від чат-бота на питання (крок 7)

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

За цей час запит доходить до моделі, вхідний промпт обробляється за один прохід і генерується перший токен. Його вимірюють окремо, бо користувач сприймає його як «паузу» перед відповіддю, і залежить він від інших чинників (мережа, довжина промпту, завантаження моделі), ніж швидкість подальшої генерації.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

До TTFT входить фіксована мережева затримка, черга й можливі приховані міркування, а після старту токени видаються швидко завдяки потужному серверному обладнанню. Саме так було на довгому промпті: TTFT 7.9 с проти 2.1 с, але 231.6 проти 107.8 симв./с.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт — стан, коли модель ще не завантажена в оперативну пам'ять: сервер мусить зчитати гігабайти ваг із диска. Наступні («теплі») запити цього не потребують. У нашому вимірі (табл. 9) холодний старт додав до TTFT ≈ 3.9 – 4.2 с — майже стільки, скільки склав load_duration.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Медіана стійка до одиничних викидів (мережевий джитер, фонові процеси). Наші повторні запуски показали розкид до 54 % у загальному часі, тож одне значення могло б легко виявитися нетиповим.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Вхідні токени обробляються паралельно за один прохід, а вихідні генеруються послідовно: кожен наступний токен залежить від усіх попередніх.

Висновок за завданням 4

Затримку не можна описувати одним числом. Локальна llama3.2:3b має майже сталий TTFT ≈ 2.1 с, але помірну швидкість генерації (до ≈ 108 симв./с), тоді як у хмарної TTFT нестабільний (1.7–9.0 с), а на довгій відповіді швидкість сягає ≈ 232 симв./с. Для інтерактивного чату критичний TTFT, для великих текстів — швидкість генерації. Повторні запуски дали розкид загального часу до 54 %, тому висновки про швидкість слід робити за медіаною кількох запусків, а не за одним виміром. Окремий фактор — холодний старт: завантаження ваг додає до TTFT локальної моделі ≈ 4 с (≈ 6.0 – 6.3 проти ≈ 2.1 с у теплому стані), тому для інтерактивного застосунку модель варто тримати завантаженою.

Завдання 5. Оцінювання якості моделей за рубрикою

Виконано через quality.py: шість завдань (тема — Київський метрополітен і перевезення вантажів), кожне двічі для кожної з трьох моделей, temperature = 0,7. Відповіді збережено у quality.json. Оцінка виставлена за відповідями з цього файлу за критеріями з табл. 10: **бал = кількість прогонів (із двох), що виконали критерій** (0, 1 або 2), максимум 12. Джерело тексту для завдання 3 — Вікіпедія, стаття «Київський метрополітен».

Таблиця 10 — Критерії оцінювання

№	Завдання	Критерій «виконано»
1	Факт	Правильний рік відкриття — 1960, без зайвого тексту.
2	Формат	Відповідь проходить json.loads() без правок (без markdown і пояснень).
3	Стиснення	Не більше двох речень і збережено всі числа з тексту (1960, 5,2 км, п'ять станцій, 1963, три лінії, 69,648 км, 52 станції, три вузли, 05:30–23:00).
4	Код	Функція total_weight коректно сумує ключ weight; лише код.
5	Логіка	Правильна відповідь — 2 рейси ($45 \cdot 8 + 75 \cdot 12 = 1260$ кг > 1200 кг).
6	Українська	Два абзаци українською, без русизмів, кальок і слів іншими мовами.

Таблиця 11 — Значення, які повернули моделі у завданнях 1 і 2 (правильно: 1960 рік, 3 лінії)

Модель	Завд. 1, прогін 1	Завд. 1, прогін 2	Завд. 2, прогін 1	Завд. 2, прогін 2
llama3.2:3b	1926	1927	year 2022, lines 6	year 1960, lines 6
qwen3:4b	1984	1984	year 1961, lines 5	year 1960, lines 5
gemini-3.8-flash	1960	1960	year 1960, lines 3	year 1960, lines 3

Таблиця 12 — Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна (gemini-3.8-flash)
1. Факт	0	0	2
2. Формат	2	2	2

Завдання	llama3.2:3b	qwen3:4b	Хмарна (gemini-3.8-flash)
3. Стиснення	0	1	2
4. Код	2	2	2
5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	4	8	12

Таблиця 13 — Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	так	ні
4. Код	ні	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	так	ні

Розбіжність — випадок, коли один прогін виконав критерій, а інший ні. У qwen3:4b у завданні 3 перший прогін пропустив «три лінії» (решта чисел на місці), другий зберіг усі числа; у завданні 6 другий прогін містить русизми («груз», «відрізняння»), перший — ні.

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше — хмарна gemini-3.8-flash (12/12), далі qwen3:4b (8/12) і llama3.2:3b (4/12). Обидві локальні моделі провалили завдання 1 (факт): llama3.2:3b назвала 1926 і 1927, qwen3:4b — 1984 обидва рази. Далі різниця виникла у завданнях 3 (стиснення), 5 (логіка) і 6 (українська), де llama3.2:3b набрала 0.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: Лише у qwen3:4b — на завданнях 3 і 6. Хмарна й llama3.2:3b були стабільними в оцінках (llama3.2:3b — стабільно слабкою). Отже, за temperature = 0,7 навіть непогана локальна модель непередбачувана від запуску до запуску, а результат одного прогону не можна вважати типовим.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, усі три моделі в обох прогонах: відповіді розпізнаються json.loads() без правок. Але формат ≠ зміст: у llama3.2:3b і qwen3:4b значення неправильні (lines_count 6 і 5 замість 3, роки 2022 і 1961), правильні значення повернула лише хмарна.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так. llama3.2:3b: слова «zawолок», «ключowych», «transported», «profil», «стовпцітий» та нісенітні конструкції. qwen3:4b: у завданні 5 (прогін 1) відповідь повністю російською, у завданні 6 (прогін 2) русизми «груз», «відрізняння», «перекладення». Хмарна модель мовних помилок не допустила.

Крок 7. Приклад провалу

Запит: «120 ящиків: 45 по 8 кг, решта по 12 кг. Вантажівка тримає 1200 кг. Скільки рейсів? Покажи хід розв'язання.» (llama3.2:3b, завдання 5, прогін 2).

Відповідь моделі: $1200 - 360 = 840$; «сумарна вага вантажівки: $1200 + 840 = 2040$ кг»; $2040 / 12 = 170$ рейсів. У прогоні 1 та сама модель отримала 16 рейсів, до того ж міркувала переважно російською.

Правильна відповідь: $45 \cdot 8 = 360$ кг, решта $120 - 45 = 75$ ящиків по 12 кг = 900 кг, разом 1260 кг; оскільки $1260 > 1200$, потрібно **2 рейси**.

Пояснення: модель не порахувала кількість «решти» ящиків, змішала місткість вантажівки з вагою вантажу й поділила довільну суму на вагу одного ящика. Хід розв'язання має правдоподібний вигляд, але не має логічного зв'язку із задачею; його переконливість не є ознакою правильності.

Відповіді від чат-бота на питання (крок 7)

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Стандартні бенчмарки (MMLU, GSM8K, HumanEval), парне порівняння відповідей людьми з рейтингом Elo (Chatbot Arena), оцінювання іншою потужною моделлю в ролі судді («LLM-as-a-judge») та автоматичні перевірки за еталоном (наприклад, unit-тести для коду).

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

У навчальних даних JSON і код майже завжди подаються в markdown-блоках, тож ця звичка глибоко закріплена. У нашому експерименті, щоправда, усі три моделі виконали вимогу, але на завданнях із коротким «чистим» виводом такий ризик залишається, тому відповідь варто валідувати кодом.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Частка україномовних текстів у навчальних корпусах значно менша, а невелика кількість параметрів обмежує здатність засвоїти менш поширені мовні патерни, тому виникають кальки, русизми й суміш мов.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Через випадковий вибір токенів при $temperature > 0$ (у нас 0,7) та дрібні відмінності в обчисленнях з плаваючою комою. Наприклад, qwen3:4b у завданні 3 в одному прогоні втратила число, в іншому — ні.

5. Що таке галюцинація мовної моделі і чому модель радше вигадас факт, ніж визнає незнання?

Галюцинація — впевнена, але хибна відповідь. Модель продовжує текст найімовірнішим способом і не звіряється з базою фактів, а прикладів чесного «не знаю» в навчальних даних мало. Приклад із роботи: обидві локальні моделі впевнено назвали хибні роки відкриття метро (1926/1927 і 1984).

Висновок за завданням 5

Фіксована рубрика з двома прогонами виразно розвела моделі: хмарна — 12/12, qwen3:4b — 8/12, llama3.2:3b — 4/12, і дозволила відрізнити стабільну поведінку від випадкової (розбіжності лише у qwen3:4b). Показово, що обидві локальні моделі не знають простого факту без опори на текст (завдання 1), а форматування JSON усі виконали правильно, але з хибним змістом. Для україномовних завдань, де важлива точність, найнадійнішою виявилась хмарна модель; qwen3:4b — прийнятний безкоштовний варіант для міркувань і стиснення наданого тексту, але не для фактів і не без перевірки мови.

Завдання 6. Керування випадковістю: temperature і top_p

Виконано через `sampling.py` на локальній моделі llama3.2:3b; скрипт запускався двічі, у таблицях наведено обидва запуски. Творче завдання: «Придумай одну назву застосунку нагадувань про ліки. Відповідай лише назвою.» Фактичне: «Рік відкриття Київського метрополітену. Відповідай лише роком.» (правильно — 1960). Для експерименту з `top_p` температура зафіксована на рівні 0,7.

Таблиця 14 — Вплив параметрів на різноманітність і правильність відповідей

Параметр	Значення	Унікальних із 5 (творче), запуск 1 / запуск 2	Правильних із 5 (фактичне)
temperature	0.0	1 / 1	0/5
temperature	0.7	5 / 5	—
temperature	1.5	5 / 5	0/5
top_p	0.1	1 / 1	—
top_p	0.5	5 / 5	—
top_p	1.0	5 / 5	—

Таблиця 15 — Фактичне питання: усі відповіді моделі (правильно — 1960)

Запуск	temperature	Відповіді (5 повторів)	Унікальних	Правильних
1	0.0	1927, 1927, 1927, 1927, 1927	1	0

Запуск	temperature	Відповіді (5 повторів)	Унікальних	Правильних
1	1.5	1927, 1927, 1935, 1927, 1935	2	0
2	0.0	1927, 1927, 1927, 1927, 1927	1	0
2	1.5	1935, 1946, 1935, 1929, 1929	3	0

Таблиця 16 — Перевірка відтворюваності (обидва запуски однакові)

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так/ні)	так

Приклади творчих відповідей. При temperature = 0.0 і top_p = 0.1 усі п'ять відповідей були однаковими — «Ліки на дудку». При 0.7 у запуску 1: «Ліки на мові», «Ликадувальник.», «Ліки для Ліжка», «Ліки на дудку.», «Ликінія». При 1.5 у запуску 1: «Skypole», «Рибіча», «Ліки Мріяву», «Liko!», «LickeySoundsQA»; у запуску 2: «СуперЛіки», «Ліковий гудбай.», «Поклоніння Лікум.», «Сканікалей», «Рикікіліці».

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: temperature = 0,0 (або близько до нуля): при 0,0 усі 10 запусків дали ідентичну відповідь, тобто вивід максимально передбачуваний, що потрібно для стабільного парсингу. Але експеримент показує й обмеження: відтворюваність не означає правильність — на фактичному питанні модель стабільно відповідала «1927» замість 1960. Значення треба перевіряти окремо.

2. Яке — генератору варіантів заголовків?

Відповідь: Приблизно 0,7–1,0. При 0,7 п'ять запусків дали п'ять різних назв, тобто різноманітності вже вистачає, а при 1,5 назви починають розпадатись на випадкові набори літер («LickeySoundsQA», «Skypole»).

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: temperature змінює форму розподілу ймовірностей для всіх кандидатів одразу (робить його гострішим або плоскішим). top_p не змінює ймовірності, а відсікає «хвіст»: лишає мінімальний набір токенів, сумарна ймовірність яких досягає p. У досліді top_p = 0,1 дав ті самі п'ять однакових відповідей, що й temperature = 0,0, — лишився фактично один найімовірніший кандидат.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: В експерименті — так: 10 запусків при temperature = 0,0 дали одну унікальну відповідь (у обох запусках скрипта; саме значення в цьому вимірі не логувалось, а в п'ятиразовому тесті це було «1927»). Теоретично ж стовідсоткової гарантії немає: на іншому обладнанні чи за паралельних обчислень з плаваючою комою результат може відрізнятись.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1,5, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: Найбезглуздіша — «LickeySoundsQA» (запуск 1): випадкова суміш латиниці й слів, що не пов'язана із застосунком для нагадувань про ліки; схожі «Skypole» і «Liko!». Висока температура сильно вирівнює розподіл, і токени з дуже малою ймовірністю (зокрема з іншого алфавіту) отримують реальний шанс бути обраними. На фактичному питанні той самий ефект дав розкид «1935», «1946», «1929» замість стабільного значення.

Відповіді від чат-бота на питання (крок 7)

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Логіти діляться на значення temperature перед softmax: при $T < 1$ різниці між ними посилюються (розподіл загострюється), при $T > 1$ — згладжуються (розподіл наближається до рівномірного); при $T \rightarrow 0$ вибір зводиться до найімовірнішого токена.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

top_p (nucleus sampling) динамічно відбирає найменший набір tokenів із сумарною ймовірністю $\geq p$ і семплює лише з нього, а temperature не змінює склад кандидатів, лише форму розподілу.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Зазвичай краще змінювати один, а другий фіксувати (часто top_p близько 1,0): їхній сумарний ефект важко передбачити, і експеримент стає менш контрольованим.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Через недетермінізм обчислень з плаваючою комою при паралельному виконанні (порядок додавання може відрізнятись), а також через різне батчування запитів на сервері.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для витягу даних, класифікації й фактичних відповідей — близько 0–0,3; для творчого письма й генерації варіантів — приблизно 0,7–1,2.

Висновок за завданням 6

temperature і top_p регулюють випадковість вибору токена, а не «розумність» моделі. На творчому завданні підвищення будь-якого з них дало різноманіття (з 1 до 5 унікальних відповідей із 5), проте на фактичному питанні випадковість шкодить: при temperature = 1,5 з'явилися різні роки (1927, 1929, 1935, 1946), а при 0,0 відповідь була стабільною — але хибною (1927 замість 1960). Отже, низька температура дає відтворюваність, а не правильність; для агентів зі структурованим виводом слід брати temperature близько 0 і додатково перевіряти результат.

Підсумкове порівняння моделей

Таблиця 17 — Порівняння локальних і хмарної моделей за даними лабораторної роботи

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна (Gemini Flash)
Швидкість генерації	14.7–107.8 симв./с (у середньому 67.7); 18.9 tokenів/с у завд. 3	30.7 tokenів/с (завд. 1); у симв./с окремо не вимірювалась	15.9–231.6 симв./с (у середньому 98.0); 114.5 tokenів/с у завд. 3
TTFT	2.09–2.12 с, майже не залежить від промпту	промпти завд. 4 не запускались; «Столиця Японії?»: 2.12 с (теплий старт)	1.70–9.03 с, нестабільний
Холодний старт (TTFT, медіана)	6.29 с проти 2.08 с у теплому (load 4.16 с)	6.00 с проти 2.12 с у теплому (load 3.86 с)	не вимірювався
Якість, балів /12	4	8	12
Задача на міркування (ручка й зошит, 5 запусків)	3/5; у середньому 146.2 tokenів, 5.03 с	5/5; у середньому 1136.6 tokenів, 39.87 с	не запускалась
Стабільність формату	JSON валідний 2/2, але зі хибними значеннями	JSON валідний 2/2, зі хибними значеннями	JSON валідний 2/2, значення правильні
completion_tokens на питання про стек і чергу	62–268	907–4071 (з прихованим thinking)	1042 (total_tokens 2037)
Потрібні ресурси	локальний ПК з Ollama	локальний ПК з Ollama; обсяг пам'яті не заміряно	лише інтернет і API-ключ
Вартість / ліміти	безкоштовно	безкоштовно	безкоштовний тариф: 5 запитів/хв, помилка 429 під час benchmark
Куди йдуть дані	залишаються на комп'ютері	залишаються на комп'ютері	надсилаються на сервери Google
Працює без інтернету	так	так	ні
Коли обирати	приватні дані, прості задачі, офлайн; не для фактів і української	міркування й стиснення наданого тексту, якщо потрібна безкоштовна локальна модель	коли критичні точність і якість мови, а приватність не головна

Загальні висновки

- Локальна й хмарна моделі працюють через єдиний OpenAI-сумісний інтерфейс, тому перехід між ними не потребує змін у логіці програми: змінюються лише `base_url`, ключ і назва моделі. Це основа резервування провайдерів.
- Затримка складається з двох незалежних показників — TTFT і швидкості генерації. Найшвидша за одним показником модель не обов'язково найшвидша за іншим: локальна виграє за TTFT на середньому й довгому промптах, хмарна — за пропускну здатністю на довгому. Окремий чинник — холодний старт: завантаження ваг додає ≈ 4 с до TTFT локальної моделі (≈ 6.0 – 6.3 проти ≈ 2.1 с у теплому стані).
- Міркувальна модель (qwen3:4b) витрачає в рази більше токенів на приховане thinking (4071 і 907 проти 251 і 62 у llama3.2:3b), але й дає значно якіснішу відповідь; system-повідомлення скорочує ці витрати, проте не виправляє нестачу знань моделі. На фактичному питанні це 133 проти 5 токенів (у ≈ 27 разів), а на задачі про ручку й зошит — у середньому 1136.6 проти 146.2 токенів, зате 5/5 правильних відповідей проти 3/5.
- Якість малої моделі на українській та на фактах недостатня: llama3.2:3b видала хибні відповіді, мовні вкраплення й нісенітні розрахунки; обидві локальні моделі не знали року відкриття метро. Надійно впоралась лише хмарна модель (12/12).
- Мовні моделі недетерміновані: між запусками виміри швидкості відрізнялись до 54 %, оцінки qwen3:4b розходились між прогонами, а низька температура дає відтворюваність, але не правильність. Тому вимірювання потребують повторів і медіани, а результати — програмної перевірки.